

HOMework #3:

Flex + Bison Parser

Version 26.03.31

Due Date: Monday April 13th, 11:59.59pm

Description:

For this assignment, you will write a **shift-reduce parser** using **FLEX** and **BISON** to recognize and validate programs written in a *subset* of the PUCK-26.1 Programming Language. Your program should output information about each token it finds and each grammar rule it reduces.

Flex tokenizer guidelines:

Your lexical analyzer should recognize the following tokens types from the PUCK-26.1 language:

- **Integers** (**T_INTEGER**) are non-empty sequences of digits optionally preceded with either a '+' or '-' sign. (e.g. 3, -12, +001, -1230).
- **Decimal** (**T_DECIMAL**) numbers are Integers followed by a period '.', followed by a non-empty sequence of digits. (e.g. 3.14, 00.01, 123.0).
- **String** literals (**T_STRING**) are sequences of **non-whitespace** characters enclosed in double quote marks (' " '). (e.g. "555.ABC.#\$%!", "ProgroTron", "Be-Happy!"). **Extra Notes:** spaces are **not** allowed on string literals. Nested double quotes are **not** allowed on string literals.
- **Identifiers** (**T_IDENT**) are strings that consist of a letter followed by zero or more letters, digits or the underscore; and that are **not** keywords. (e.g. x, size, name, p3, r_val).
- **Keywords** are specific strings that form the language. For this homework each keyword will have its own token type. The following strings are keywords: 'IF', 'THEN', 'ELSE', ';', 'WHILE', 'END', '(', ')', and 'WRITE'. Flex shall recognize each of these as token types **K_IF**, **K_THEN**, **K_ELSE**, **K_SC**, **K_WHILE**, **K_END**, **K_LPAREN**, **K_RPAREN** and **K_WRITE**.

- **Operators** are special strings that flex should recognize in the following way:
 - **OP_ASSIGN** : the string '='.
 - **OP_NEG** : the string '~'.
 - **OP_RELATION** : the strings '<', '>', '=', and '#'.
 - **OP_ADD** : the strings '+', '-', '&' and 'OR'.
 - **OP_MULT** : the strings '*', '/' and 'AND'.

Whenever your tokenizer finds and returns a token it shall print a message of the form:

```
TOKEN: <TokenType>      LEXEME : <Lexeme>
```

Reminder: In PUCK-26.1 tokens are always separated by white-spaces.

Bison Parser Guidelines:

Your **Bison** parser will use the following grammar that specifies a *subset* of the PUCK-26.1 programming language.

The grammar is in **BNF** and "StatementSequence" is the start symbol. Terminal symbols are in **bold**. Collections of terminal symbols are in **blue**.

```

Expression ::= SimpleExpression | SimpleExpression OP_RELATION Expression
SimpleExpression ::= Term | Term OP_ADD SimpleExpression
Term ::= Factor | Factor OP_MULT Term
Factor ::= T_INTCONST | T_DECCONST | T_STRCONST | T_IDENT | ( Expression ) | ~ Factor

Assignment ::= T_IDENT := Expression
WriteStatement ::= WRITE ( Expression )
IfStatement ::= IF Expression THEN StatementSequence END |
               IF Expression THEN StatementSequence ELSE StatementSequence END
WhileStatement ::= WHILE Expression DO StatementSequence END

Statement ::= Assignment | WriteStatement | IfStatement | WhileStatement
StatementSequence ::= Statement | Statement ; StatementSequence
  
```

Whenever Bison is able to match a rule, it shall output a message of the format:

```
RULE: <Head> ::= <BODY>
```

As much as possible, try to match the format of the sample input output.

Input:

Your Parser should accept input from **standard input** (keyboard) and output to **standard output** (display).

Output:

The output of your lexical analyzer should **match** the format of the sample output, [although whitespace will be ignored when grading](#).

Your output should end with the word "VALID" if the input is a correct program under the grammar, and "INVALID!" if it is not.

Submission:

Submit your assignment by placing all code/scripts in this assignment's git repository in the course's GitLab server, [link](#) . (You should have a repository setup this weekend).

You will submit two files that shall be called “puckparser.l” and “puckparser.y”. Your files should include your name. Your files will be compiled, run, tested and graded using the following chain of commands:

```
bison -d puckparser.y
flex puckparser.l
g++ puckparser.tab.c lex.yy.c -o puckparser.ex
./puckparser.ex < inputFileNames
```

Your program will be evaluated and graded on the Computer Science department's Linux machines so your program needs to be compatible with the current system, compilers and environment.

Sample 1:

Input	Output
x := 2 + 2 ; WRITE (x * 100)	TOKEN: T_IDENT LEXEME: x TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_INTEGER LEXEME: 2 RULE: Factor ::= T_INTEGER TOKEN: OP_ADD LEXEME: +

	<pre> RULE: Term ::= Factor TOKEN: T_INTEGER LEXEME: 2 RULE: Factor ::= T_INTEGER TOKEN: K_SC LEXEME: ; RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: SimpleExpression ::= Term OP_ADD SimpleExpression RULE: Expression ::= SimpleExpression RULE: Assignment ::= identifier := Expression RULE: Statement ::= Assignment TOKEN: K_WRITE LEXEME: WRITE TOKEN: K_LPAREN LEXEME: (TOKEN: T_IDENT LEXEME: x RULE: Factor ::= T_IDENT TOKEN: OP_MULT LEXEME: * TOKEN: T_INTEGER LEXEME: 100 RULE: Factor ::= T_INTEGER TOKEN: K_RPAREN LEXEME:) RULE: Term ::= Factor RULE: Term ::= Factor OP_MULT Term RULE: SimpleExpression ::= Term RULE: Expression ::= SimpleExpression RULE: WriteStatement ::= WRITE (Expression) RULE: Statement ::= WriteStatement RULE: StatementSequence ::= Statement RULE: StatementSequence ::= Statement ; StatementSequence VALID </pre>
--	--

Sample 2:

Input	Output
<pre> x := 1 ; y := 2 ; c := 3 ; WHILE c < n DO x := x + y ; y := x - y ; c := c + 1 END ; WRITE (x) </pre>	<pre> TOKEN: T_IDENT LEXEME: x TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_INTEGER LEXEME: 1 RULE: Factor ::= T_INTEGER TOKEN: K_SC LEXEME: ; RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: Expression ::= SimpleExpression RULE: Assignment ::= identifier := Expression RULE: Statement ::= Assignment TOKEN: T_IDENT LEXEME: y TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_INTEGER LEXEME: 2 RULE: Factor ::= T_INTEGER TOKEN: K_SC LEXEME: ; </pre>

```

RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: Expression ::= SimpleExpression
RULE: Assignment ::= identifier := Expression
RULE: Statement ::= Assignment
TOKEN: T_IDENT      LEXEME: c
TOKEN: OP_ASSIGN    LEXEME: :=
TOKEN: T_INTEGER    LEXEME: 3
RULE: Factor ::= T_INTEGER
TOKEN: K_SC         LEXEME: ;
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: Expression ::= SimpleExpression
RULE: Assignment ::= identifier := Expression
RULE: Statement ::= Assignment
TOKEN: K_WHILE      LEXEME: WHILE
TOKEN: T_IDENT      LEXEME: c
RULE: Factor ::= T_IDENT
TOKEN: OP_RELATION  LEXEME: <
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
TOKEN: T_IDENT      LEXEME: n
RULE: Factor ::= T_IDENT
TOKEN: K_DO         LEXEME: DO
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: Expression ::= SimpleExpression
RULE: Expression ::= SimpleExpression
OP_RELATION Expression
TOKEN: T_IDENT      LEXEME: x
TOKEN: OP_ASSIGN    LEXEME: :=
TOKEN: T_IDENT      LEXEME: x
RULE: Factor ::= T_IDENT
TOKEN: OP_ADD       LEXEME: +
RULE: Term ::= Factor
TOKEN: T_IDENT      LEXEME: y
RULE: Factor ::= T_IDENT
TOKEN: K_SC         LEXEME: ;
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: SimpleExpression ::= Term OP_ADD
SimpleExpression
RULE: Expression ::= SimpleExpression
RULE: Assignment ::= identifier := Expression
RULE: Statement ::= Assignment
TOKEN: T_IDENT      LEXEME: y
TOKEN: OP_ASSIGN    LEXEME: :=
TOKEN: T_IDENT      LEXEME: x
RULE: Factor ::= T_IDENT
TOKEN: OP_ADD       LEXEME: -
RULE: Term ::= Factor
TOKEN: T_IDENT      LEXEME: y
RULE: Factor ::= T_IDENT

```

```

TOKEN: K_SC          LEXEME: ;
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: SimpleExpression ::= Term OP_ADD
SimpleExpression
RULE: Expression ::= SimpleExpression
RULE: Assignment ::= identifier := Expression
RULE: Statement ::= Assignment
TOKEN: T_IDENT       LEXEME: c
TOKEN: OP_ASSIGN      LEXEME: :=
TOKEN: T_IDENT       LEXEME: c
RULE: Factor ::= T_IDENT
TOKEN: OP_ADD         LEXEME: +
RULE: Term ::= Factor
TOKEN: T_INTEGER      LEXEME: 1
RULE: Factor ::= T_INTEGER
TOKEN: K_END          LEXEME: END
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: SimpleExpression ::= Term OP_ADD
SimpleExpression
RULE: Expression ::= SimpleExpression
RULE: Assignment ::= identifier := Expression
RULE: Statement ::= Assignment
RULE: StatementSequence ::= Statement
RULE: StatementSequence ::= Statement ;
StatementSequence
RULE: StatementSequence ::= Statement ;
StatementSequence
RULE: WhileStatement ::= WHILE Expression DO
StatementSequence END
RULE: Statement ::= WhileStatement
TOKEN: K_SC          LEXEME: ;
TOKEN: K_WRITE       LEXEME: WRITE
TOKEN: K_LPAREN      LEXEME: (
TOKEN: T_IDENT       LEXEME: x
RULE: Factor ::= T_IDENT
TOKEN: K_RPAREN      LEXEME: )
RULE: Term ::= Factor
RULE: SimpleExpression ::= Term
RULE: Expression ::= SimpleExpression
RULE: WriteStatement ::= WRITE ( Expression )
RULE: Statement ::= WriteStatement
RULE: StatementSequence ::= Statement
RULE: StatementSequence ::= Statement ;
StatementSequence
RULE: StatementSequence ::= Statement ;
StatementSequence
RULE: StatementSequence ::= Statement ;
StatementSequence
VALID

```

Sample 3:

Input	Output
<pre> WHILE a # b DO IF a > b THEN a := a - b ELSE b := b - a END END ; WRITE ("Hello" & "World!") </pre>	<pre> TOKEN: K_WHILE LEXEME: WHILE TOKEN: T_IDENT LEXEME: a RULE: Factor ::= T_IDENT TOKEN: OP_RELATION LEXEME: # RULE: Term ::= Factor RULE: SimpleExpression ::= Term TOKEN: T_IDENT LEXEME: b RULE: Factor ::= T_IDENT TOKEN: K_DO LEXEME: DO RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: Expression ::= SimpleExpression RULE: Expression ::= SimpleExpression OP_RELATION Expression TOKEN: K_IF LEXEME: IF TOKEN: T_IDENT LEXEME: a RULE: Factor ::= T_IDENT TOKEN: OP_RELATION LEXEME: > RULE: Term ::= Factor RULE: SimpleExpression ::= Term TOKEN: T_IDENT LEXEME: b RULE: Factor ::= T_IDENT TOKEN: K_THEN LEXEME: THEN RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: Expression ::= SimpleExpression RULE: Expression ::= SimpleExpression OP_RELATION Expression TOKEN: T_IDENT LEXEME: a TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_IDENT LEXEME: a RULE: Factor ::= T_IDENT TOKEN: OP_ADD LEXEME: - RULE: Term ::= Factor TOKEN: T_IDENT LEXEME: b RULE: Factor ::= T_IDENT TOKEN: K_ELSE LEXEME: ELSE RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: SimpleExpression ::= Term OP_ADD SimpleExpression RULE: Expression ::= SimpleExpression RULE: Assignment ::= identifier := Expression RULE: Statement ::= Assignment RULE: StatementSequence ::= Statement TOKEN: T_IDENT LEXEME: b TOKEN: OP_ASSIGN LEXEME: := </pre>

	TOKEN: T_IDENT LEXEME: b RULE: Factor ::= T_IDENT TOKEN: OP_ADD LEXEME: - RULE: Term ::= Factor TOKEN: T_IDENT LEXEME: a RULE: Factor ::= T_IDENT TOKEN: K_END LEXEME: END RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: SimpleExpression ::= Term OP_ADD SimpleExpression RULE: Expression ::= SimpleExpression RULE: Assignment ::= identifier := Expression RULE: Statement ::= Assignment RULE: StatementSequence ::= Statement RULE: IfStatement ::= IF Expression THEN StatementSequence ELSE StatementSequence END RULE: Statement ::= IfStatement TOKEN: K_END LEXEME: END RULE: StatementSequence ::= Statement RULE: WhileStatement ::= WHILE Expression DO StatementSequence END RULE: Statement ::= WhileStatement TOKEN: K_SC LEXEME: ; TOKEN: K_WRITE LEXEME: WRITE TOKEN: K_LPAREN LEXEME: (TOKEN: T_STRING LEXEME: "Hello" RULE: Factor ::= T_STRING TOKEN: OP_ADD LEXEME: & RULE: Term ::= Factor TOKEN: T_STRING LEXEME: "World!" RULE: Factor ::= T_STRING TOKEN: K_RPAREN LEXEME:) RULE: Term ::= Factor RULE: SimpleExpression ::= Term RULE: SimpleExpression ::= Term OP_ADD SimpleExpression RULE: Expression ::= SimpleExpression RULE: WriteStatement ::= WRITE (Expression) RULE: Statement ::= WriteStatement RULE: StatementSequence ::= Statement RULE: StatementSequence ::= Statement ; StatementSequence VALID
--	--

Sample 4:

Input	Output
<pre>void main () { cout << "Hello World"; }</pre>	<pre>TOKEN: T_IDENT LEXEME: void TOKEN: T_IDENT LEXEME: main ERROR: In line 1 with token 'main' INVALID!</pre>

Sample 5:

Input	Output
<pre>WRITE WRITE ; x := WHILE ; WRITE (:) ; IF</pre>	<pre>TOKEN: K_WRITE LEXEME: WRITE TOKEN: K_WRITE LEXEME: WRITE ERROR: In line 1 with token 'WRITE' INVALID!</pre>

Hint Code:

The following code recognizes the following *tiny subset* of the assignment's language:

Assignment := **T_IDENT** := **T_DECCONST**
WriteStatement := **WRITE** (**T_INTCONST**)
Statement := Assignment | WriteStatement
StatementSequence = Statement | Statement ; StatementSequence

hint.y

```
/* *****  
 * hint.y  
 * ***** */  
// -- PREAMBLE -----  
%{  
#include <iostream>  
using namespace std;  
  
    // Things from Flex that Bison needs to know  
extern int yylex();  
extern int line_num;  
extern char* yytext;  
  
    // Prototype for Bison's error message function  
int yyerror(const char *p);  
%}  
  
//-- TOKEN DEFINITIONS --  
// what tokens to expect from Flex  
  
%token K_SC  
%token K_LPAREN  
%token K_RPAREN  
%token K_WRITE  
  
%token OP_ASSIGN  
  
%token T_IDENT  
%token T_INTEGER  
%token T_DECIMAL  
  
%%  
  
//-- GRAMMAR RULES -----  
/* NOTE: Bison likes the start symbol to be the first rule */  
StatementSequence : Statement  
                  { cout << "RULE: StatementSequence ::= Statement" << endl; }  
                  | Statement K_SC StatementSequence  
                  { cout << "RULE: StatementSequence ::= Statement ;  
StatementSequence" << endl; }
```

```

Statement : Assignment
          { cout << "RULE: Statement ::= Assignment" << endl; }
        | WriteStatement
          { cout << "RULE: Statement ::= WriteStatement" << endl; }
        ;

Assignment : T_IDENT OP_ASSIGN T_DECIMAL
          { cout << "RULE: Assignment ::= T_IDENT := T_DECIMAL" << endl; }
        ;

WriteStatement : K_WRITE K_LPAREN T_INTEGER K_RPAREN
              { cout << "RULE: WriteStatement ::= WRITE ( T_INTEGER )" << endl; }
            ;

%% //-- EPILOGUE -----
// Bison error function
int yyerror(const char *p)
{
    cout << "ERROR: In line " << line_num << " with token \"
        << yytext << "\" << endl;
    return 0;
}

int main()
{
    int failcode;
    cout << "Hello Flex + Bison" << endl;
    failcode = yyparse();

    if (failcode)
        cout << "INVALID!" << endl;
    else
        cout << "CORRECT" << endl;
    return 0;
}

```

hint.l

```

/*****
 * hint.l
 *****/
/* -- PREAMBLE ----- */
%{
#include <iostream>
#include <iomanip>
#include "hint.tab.h"      // file generated by Bison
using namespace std;

int line_num = 1;        // to keep line numbers

// Function prototype
void writeToken(const char* toktype);

%}

```

```

/* -- OPTIONS and DEFINITIONS ----- */
%option noyywrap

WHITE      [ \t]
DIGIT      [0-9]
ALPHA      [a-zA-Z]
ALPHANUM   [a-zA-Z0-9]

%% /* -- REGULAR EXPRESSIONS ----- */

{WHITE}    ; /* eat white space */
\n         { line_num++; } /* count line numbers */

/* Keyword tokens */
";"        { writeToken("K_SC"); return K_SC; }
"("        { writeToken("K_LPAREN"); return K_LPAREN; }
")"        { writeToken("K_RPAREN"); return K_RPAREN; }
"WRITE"    { writeToken("K_WRITE"); return K_WRITE; }

/* Operator tokens */
":="       { writeToken("OP_ASSIGN"); return OP_ASSIGN; }

/* Complex Tokens */
{ALPHA}{ALPHANUM}* { writeToken("T_IDENT"); return T_IDENT; }
[+-]?{DIGIT}+      { writeToken("T_INTEGER"); return T_INTEGER; }
[+-]?{DIGIT}+\.{DIGIT}+ { writeToken("T_DECIMAL"); return T_DECIMAL; }

%% /* -- EPILOGUE ----- */

// Purpose: prints that a token was found
void writeToken(const char* toktype)
{
    cout << "TOKEN: ";
    cout << left << setw(14) << toktype;
    cout << "LEXEME: " << yytext << endl;
}

```

Hint Code input and output:

Hint Sample 1:

Input	Output
apple := 13.5 ; banana := 3.14 ; WRITE (123)	Hello Flex + Bison TOKEN: T_IDENT LEXEME: apple TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_DECIMAL LEXEME: 13.5 RULE: Assignment ::= T_IDENT := T_DECIMAL RULE: Statement ::= Assignment TOKEN: K_SC LEXEME: ;

	TOKEN: T_IDENT LEXEME: banana TOKEN: OP_ASSIGN LEXEME: := TOKEN: T_DECIMAL LEXEME: 3.14 RULE: Assignment ::= T_IDENT := T_DECIMAL RULE: Statement ::= Assignment TOKEN: K_SC LEXEME: ; TOKEN: K_WRITE LEXEME: WRITE TOKEN: K_LPAREN LEXEME: (TOKEN: T_INTEGER LEXEME: 123 TOKEN: K_RPAREN LEXEME:) RULE: WriteStatement ::= WRITE (T_INTEGER) RULE: Statement ::= WriteStatement RULE: StatementSequence ::= Statement RULE: StatementSequence ::= Statement ; StatementSequence RULE: StatementSequence ::= Statement ; StatementSequence CORRECT
--	---

Hint Sample 2:

Input	Output
WRITE (123) ; PRINT (123)	Hello Flex + Bison TOKEN: K_WRITE LEXEME: WRITE TOKEN: K_LPAREN LEXEME: (TOKEN: T_INTEGER LEXEME: 123 TOKEN: K_RPAREN LEXEME:) RULE: WriteStatement ::= WRITE (T_INTEGER) RULE: Statement ::= WriteStatement TOKEN: K_SC LEXEME: ; TOKEN: T_IDENT LEXEME: PRINT TOKEN: K_LPAREN LEXEME: (ERROR: In line 2 with token '(' INVALID!

END.